

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2

Группа: Р3407

Выполнил:

Батманов Д.Е.

Проверил:

к.т.н. преподаватель

Белозубов А.В.

Санкт-Петербург

2026г.

Оглавление	
<u>ВВЕДЕНИЕ</u>	3
<u>ВЫПОЛНЕНИЕ</u>	5
<u>ЗАКЛЮЧЕНИЕ</u>	23

ВВЕДЕНИЕ

Лабораторная работа содержала три основных блока: работа с Gnome Boxes для освоения базовых принципов, ручное создание VM через QEMU для понимания низкоуровневых механизмов, и использование Virtual Machine Manager с утилитами libvirt для профессионального управления виртуальными машинами.

Работа выполнялась в трехуровневой архитектуре виртуализации: физическая система с гипервизором Oracle VirtualBox, гостевая операционная система Ubuntu, внутри которой разворачивались вложенные виртуальные машины на базе QEMU/KVM.

QEMU (Quick Emulator) — программный эмулятор процессоров и виртуальных устройств. При совместной работе с модулем ядра KVM (Kernel-based Virtual Machine) обеспечивает производительность, близкую к нативной, за счет использования аппаратных расширений виртуализации процессора.

libvirt — унифицированная библиотека управления виртуализацией, предоставляющая единый API для работы с различными гипервизорами. Включает демон libvirtd и набор консольных утилит для автоматизации управления виртуальными системами.

В процессе работы использовались три подхода к созданию виртуальных машин:

Gnome Boxes — графический интерфейс для быстрого развертывания VM с минимальной конфигурацией, позволяющий запускать системы в Live-режиме без установки на диск.

QEMU в режиме прямого запуска — низкоуровневый подход с ручным указанием всех параметров через командную строку, обеспечивающий детальный контроль над конфигурацией виртуального оборудования.

Virtual Machine Manager (virt-manager) — полнофункциональный графический менеджер, предоставляющий доступ ко всем возможностям libvirt: управление виртуальным оборудованием, мониторинг ресурсов в реальном времени, работа со снимками состояния.

Консольные утилиты libvirt включают virsh (управление VM), virt-install (автоматизированное создание), virt-clone (клонирование), osinfo-query (информация о поддерживаемых ОС).

В рамках работы выполнялось конфигурирование вложенной виртуализации с активацией Intel VT-x в BIOS и включением Nested VT-x в настройках VirtualBox. Для размещения образов подключался дополнительный виртуальный диск объемом 30 ГБ.

Изучались механизмы моментальных снимков (snapshots), позволяющие сохранять состояние виртуальной машины с возможностью последующего восстановления, а также процедуры клонирования VM посредством утилиты virt-clone для создания шаблонных систем.

Практическая часть включала настройку удаленных RDP-подключений из вложенных виртуальных машин к физическому компьютеру через проброс портов с использованием утилиты socat для перенаправления трафика между изолированными сетевыми сегментами.

Цель работы

Целью данной лабораторной работы является освоение технологий виртуализации в Unix-подобных операционных системах, изучение различных подходов к созданию и управлению виртуальными машинами на базе QEMU/KVM и библиотеки libvirt, а также получение практических навыков работы с механизмами вложенной виртуализации.

Задачи работы

1. Настроить окружение для вложенной виртуализации с активацией Nested VT-x.
2. Сконфигурировать дополнительное дисковое пространство для размещения образов виртуальных машин.
3. Освоить три подхода к созданию VM: Gnome Boxes, прямая работа с QEMU/KVM, Virtual Machine Manager.
4. Изучить механизмы создания снимков состояния и клонирования виртуальных машин.
5. Освоить консольные утилиты управления виртуализацией: virsh, virt-install, virt-clone.
6. Настроить удаленные подключения к виртуальным машинам через протокол RDP.

ВЫПОЛНЕНИЕ

1. Настройка системы

В гостевую ОС Ubuntu был добавлен и смонтирован новый жесткий диск SATA объемом 30 ГБ (Рисунок 1 и Рисунок 2)

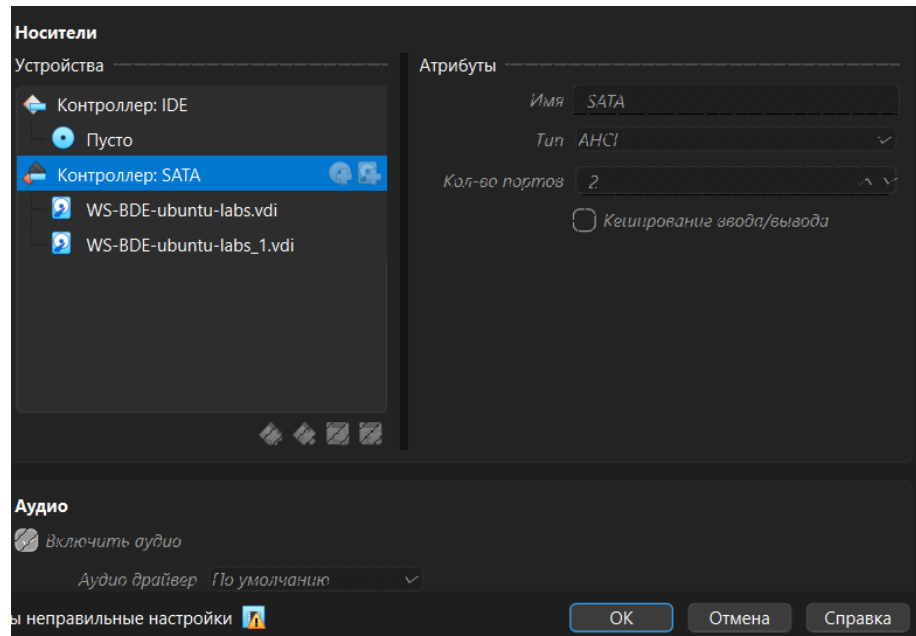


Рисунок 1 – Добавление нового жесткого диска

```
sudo fdisk /dev/sdb
```

```
sudo mkfs.ext4 /dev/sdb1
```

```
sudo mkdir ~/data
```

```
sudo mount /dev/sdb1 ~/data
```

```
sudo blkid /dev/sdb1
```

```
sudo nano /etc/fstab
```

Добавляем строку

```
UUID=0b1fd477-2d67-472b-bf5c-3233e3c3cc7c
```

```
/home/a1/data ext4 defaults 0 2
```

```
sudo mount -a
```

```
a1@a1:~$ df -h | grep data
/dev/sdb1      35G   24K   33G   1% /home/a1/data
```

Рисунок 2 – Проверка корректности монтирования диска в системе

Также была проверена конфигурация системы в Virtual Box (Рисунок 3 и Рисунок 4).

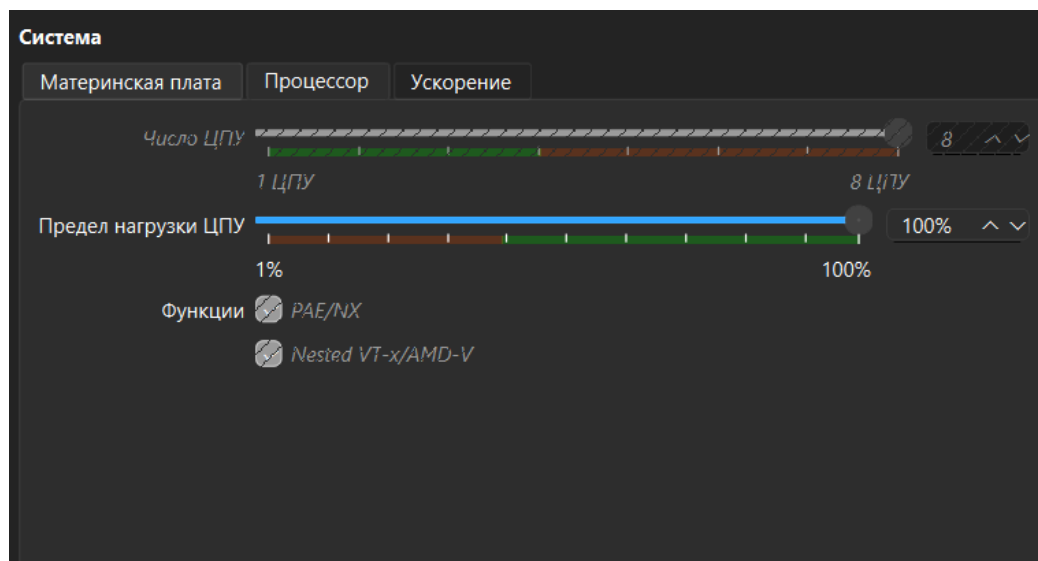


Рисунок 3 – Конфигурация системы

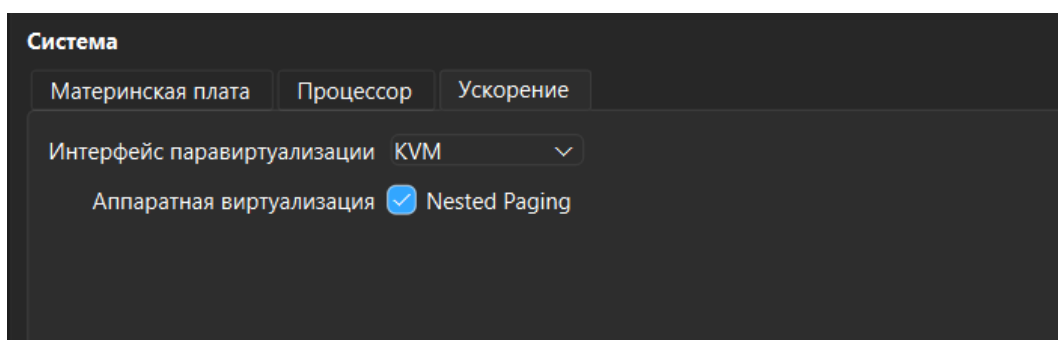


Рисунок 4 – Конфигурация системы

2. Работа с Gnome Boxes

В гостевой ОС было установлено приложение Gnome Boxes, в нем сконфигурирована новая виртуальная машина (Рисунок 5).

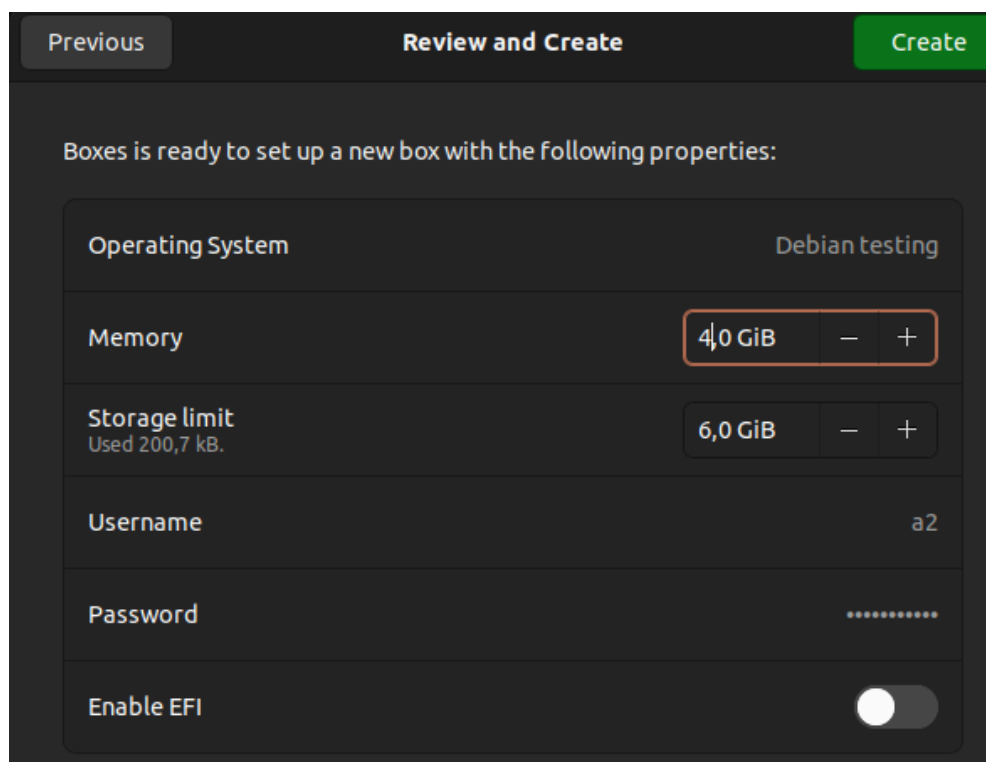


Рисунок 5 – Конфигурация виртуальной машины

Добавляем новую виртуальную машину (Рисунок 6).

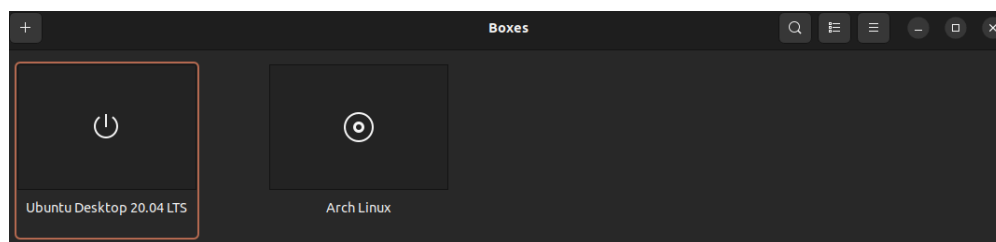


Рисунок 6 – Созданная ВМ в приложении

Запускаем вложенную ОС в режиме LiveCD (Рисунок 7):

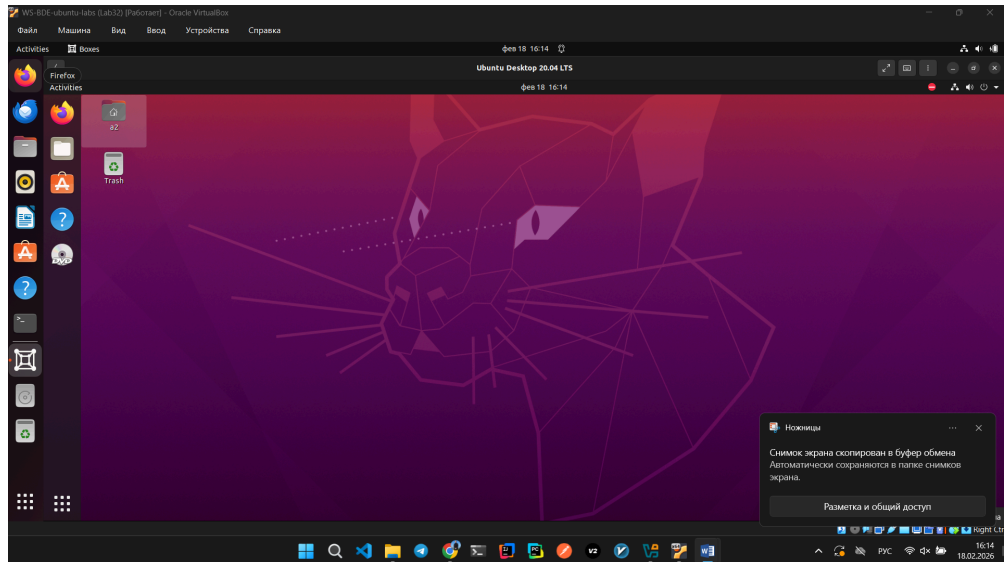


Рисунок 7 – Запущенная ВМ в GNOME Boxes

Проанализировали параметры, доступные для настройки в Boxes (Рисунок 8):

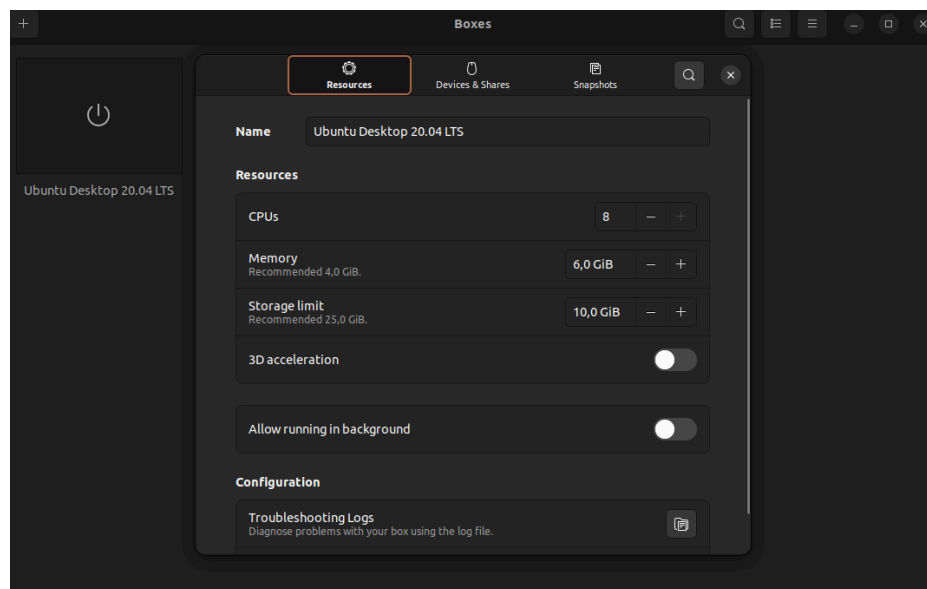


Рисунок 8 – Доступные параметры

Создаем на вложенной гостевой ОС удаленное подключение к серверу helios (Рисунок 9):

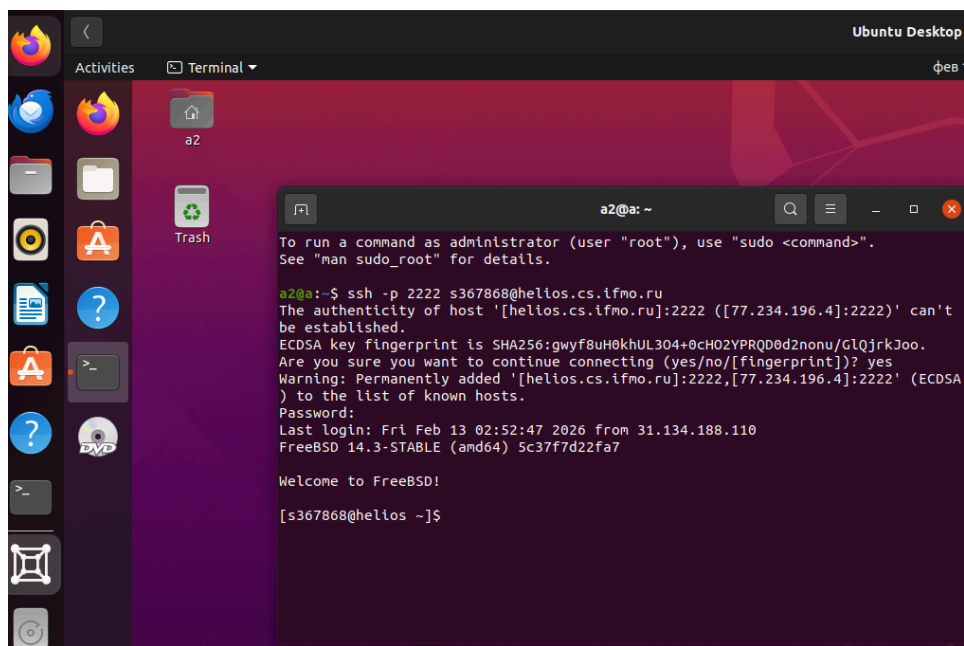


Рисунок 9 – Создание удаленного подключения к helios

Выставляем в настройках гостевой ОС 1 сетевой мост (Рисунок 10):

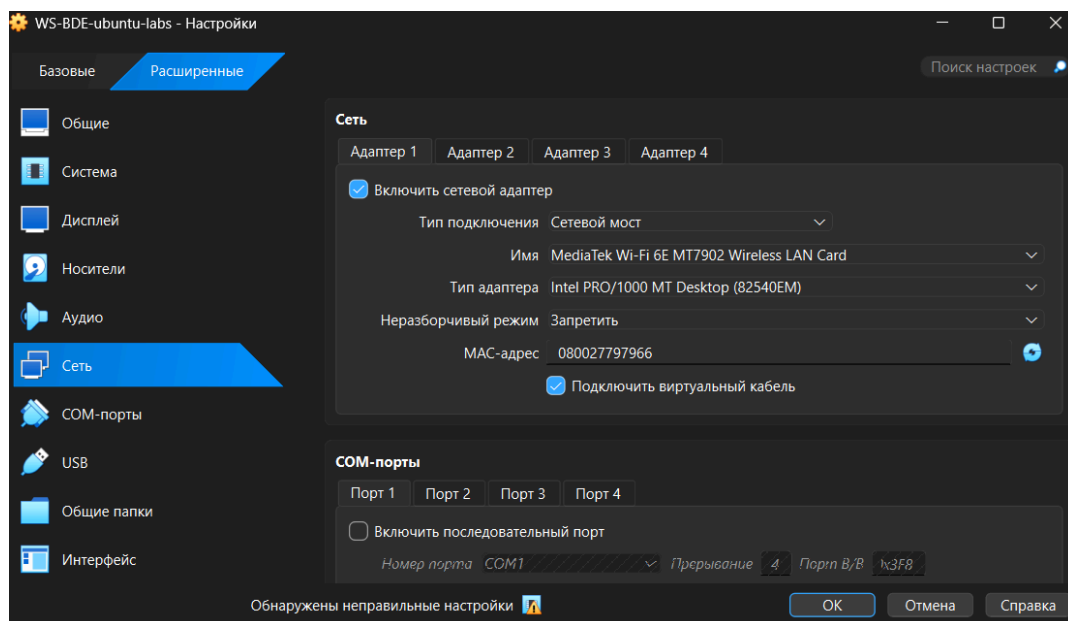


Рисунок 10 – Настройка сетевого моста

Активируем на целевом компьютере доступ по РДП (находится в одной сети с ВМ для избежания необходимости получения статического IP), проверяем доступность порта и соединений (Рисунок 11):

```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> Get-Service TermService

Status      Name             DisplayName
-----
Running     TermService      Службы удаленных рабочих столов

PS C:\WINDOWS\system32> netstat -an | finstr 3389
finstr : Имя "finstr" не распознано как имя командлета, функции, файла сценария или выполняемой
команды. Проверьте правильность написания имени, а также наличие и правильность пути, после чего повторите попытку.
строка:1 знак:15
+ netstat -an | finstr 3389
~~~~~~
+ CategoryInfo          : ObjectNotFound: (finstr:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

PS C:\WINDOWS\system32> netstat -an | findstr 3389
TCP    0.0.0.0:3389        0.0.0.0:0          LISTENING
TCP    [::]:3389         [::]:0             LISTENING
UDP    0.0.0.0:3389        *.*                *.*
UDP    [::]:3389         [::]:*              *.*
PS C:\WINDOWS\system32>
```

Рисунок 11 – Проверка доступности порта на Windows

На гостевой ОС 1 (Ubuntu) настраиваем перенаправление и проброс портов через socat для взаимодействия между гостевой ОС 2 и физической сетью (Рисунок 12):

```
a1@a1:~$ sudo socat -d -d TCP-LISTEN:3390,fork,reuseaddr TCP:192.168.0.105:3389
[5] 2964
a1@a1:~$ 2026/02/19 02:19:18 socat[2966] W ioctl(5, IOCTL_VM_SOCKETS_GET_LOCAL_CID, ...): Inappropriate ioctl for device
2026/02/19 02:19:18 socat[2966] N listening on AF=2 0.0.0.0:3390
2026/02/19 02:19:22 socat[2966] N accepting connection from AF=2 127.0.0.1:57362
on AF=2 127.0.0.1:3390
```

Рисунок 12 – Настройка проброса портов через socat

На гостевой ОС 2 проверяем настройки гетвея (Рисунок 13):

```
user@debian:~$ ip route show
default via 10.0.2.2 dev enp1s0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev enp1s0 proto kernel scope link src 10.0.2.15 metric 100

a2@a:~$ ip route show
default via 10.0.2.2 dev enp1s0 proto dhcp metric 100
10.0.2.0/24 dev enp1s0 proto kernel scope link src 10.0.2.15 metric 100
169.254.0.0/16 dev enp1s0 scope link metric 1000
a2@a:~$
```

Рисунок 13 – Поиск адреса гетвея

Устанавливаем на гостевую ОС 2 RDP клиент Remmina. Через него настраиваем подключение к удаленному рабочему столу, используя гетвей (Рисунок 14):

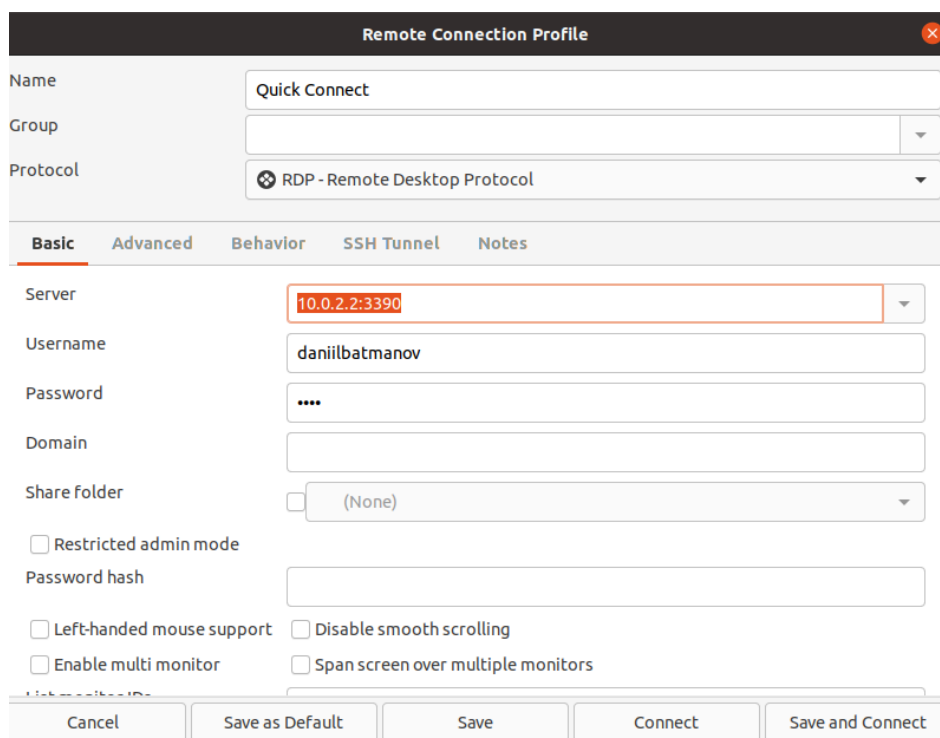


Рисунок 14 – Настройка RDP подключения в Remmina

Финальным этапом подключаемся к целевой машине на Windows (Рисунок 15):

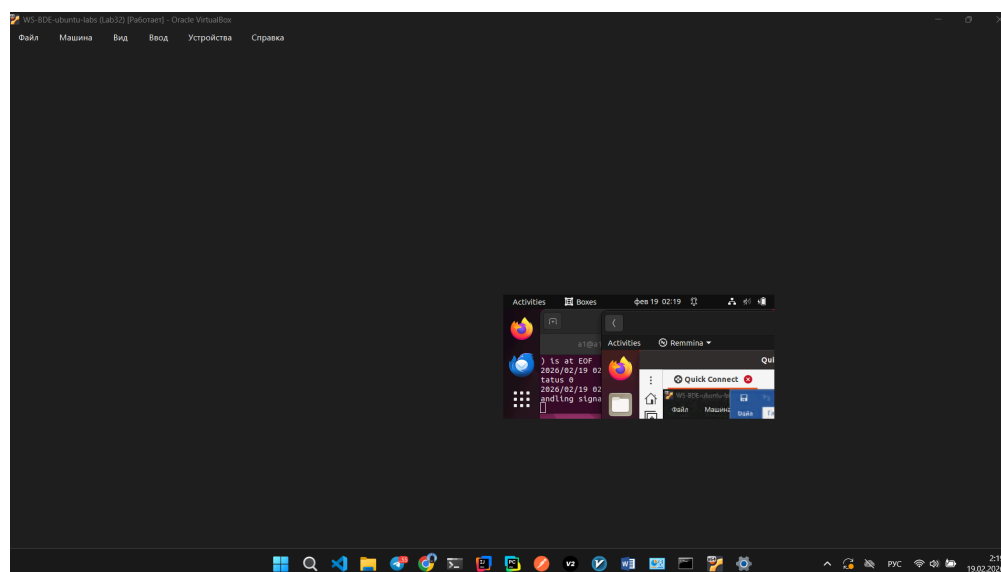


Рисунок 15 – Работающее подключение к удаленному рабочему столу
Подключение работает корректно. Только разрешение сходит с ума.

3. Работа с VMM

Была выполнена установка QEMU и проверена поддержка системой KVM, после чего была создана виртуальная машина с использованием Fedora 36 (Рисунок 16).

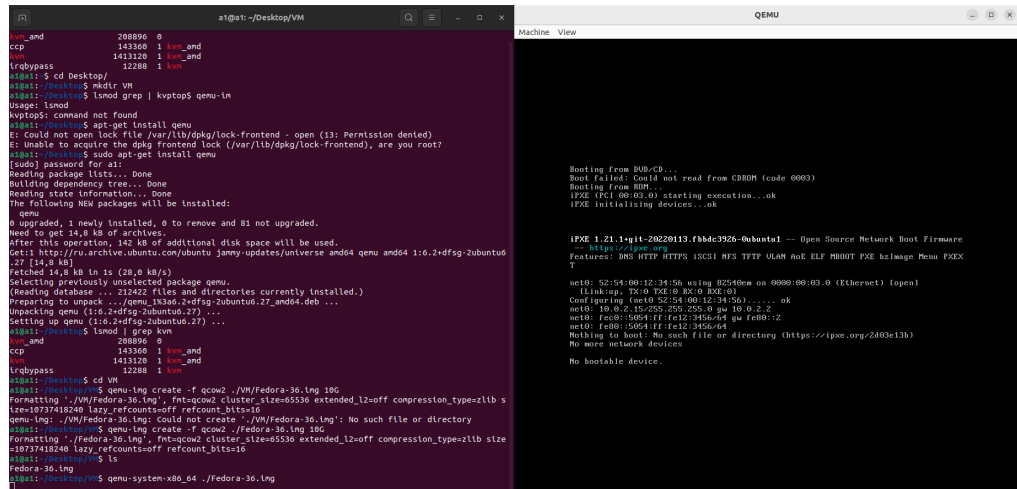


Рисунок 16 – Установка инструментов и создание ВМ

Далее был установлен iso образ нужной версии операционной системы и запущена ВМ с установкой системы (Рисунок 17):

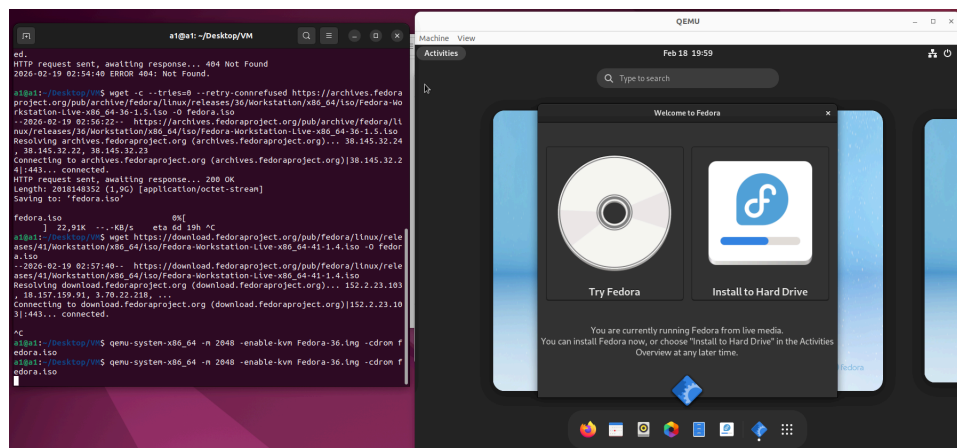


Рисунок 17 – Запуск ВМ с установкой системы

Создаем команду запуска гостевой ОС с нужными параметрами (для полноты отчета приведена команда для запуска без установки системы, для запуска после установки можно убрать содержимое после mac адреса):

```
qemu-system-x86_64 -m 4096 -enable-kvm -full-screen -netdev  
user,id=net0 -device  
virtio-net-pci,netdev=net0,mac=17:10:20:22:20:09 Fedora-36.img  
-cdrom fedora.iso
```

4. Установка Virtual Man

Были установлены нужные пакеты приложений, настроены группы пользователя и проверена корректность работы libvirt (Рисунок 18).

```
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for dbus (1.12.20-2ubuntu4.1) ...
a1@a1:~/Desktop/VM$ sudo gpasswd -a $USER libvirt
Adding user a1 to group libvirt
a1@a1:~/Desktop/VM$ sudo systemctl status libvirtd
● libvirtd.service - Virtualization daemon
   Loaded: loaded (/lib/systemd/system/libvirtd.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-02-19 04:25:46 MSK; 1min 15s ago
     TriggeredBy: ● libvirtd-admin.socket
                  ● libvirtd.socket
                  ● libvirtd-ro.socket
        Docs: man:libvirtd(8)
              https://libvirt.org
       Main PID: 3306 (libvirtd)
         Tasks: 21 (limit: 32768)
        Memory: 15.9M
          CPU: 623ms
        CGroup: /system.slice/libvirtd.service
                └─3306 /usr/sbin/libvirtd
                  └─3431 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default
                    └─3432 /usr/sbin/dnsmasq --conf-file=/var/lib/libvirt/dnsmasq/default

feb 19 04:25:46 a1 systemd[1]: Started Virtualization daemon.
```

Рисунок 18 – Настройка системы для работы virtual manager

Также была проверена правильность работы системы и запущено приложение Virtual Machine Manager (Рисунок 19):

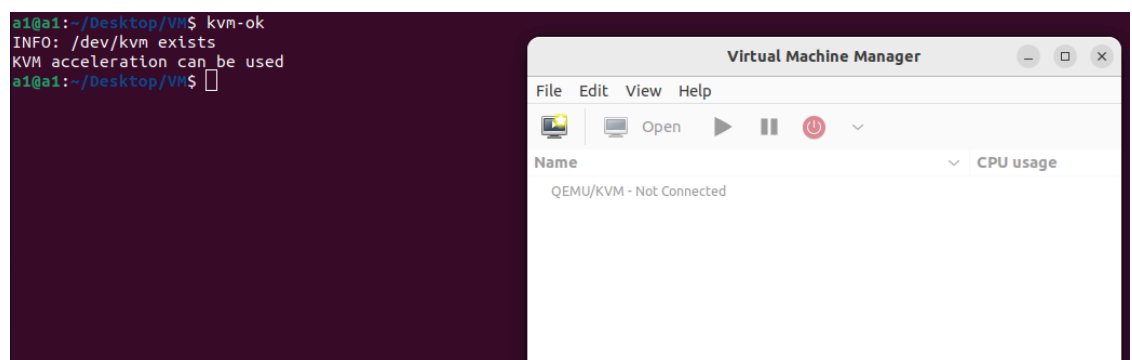


Рисунок 19 – Запуск VMM

Далее была создана и запущена виртуальная машина на базе Fedora (Рисунок 20 и Рисунок 21):

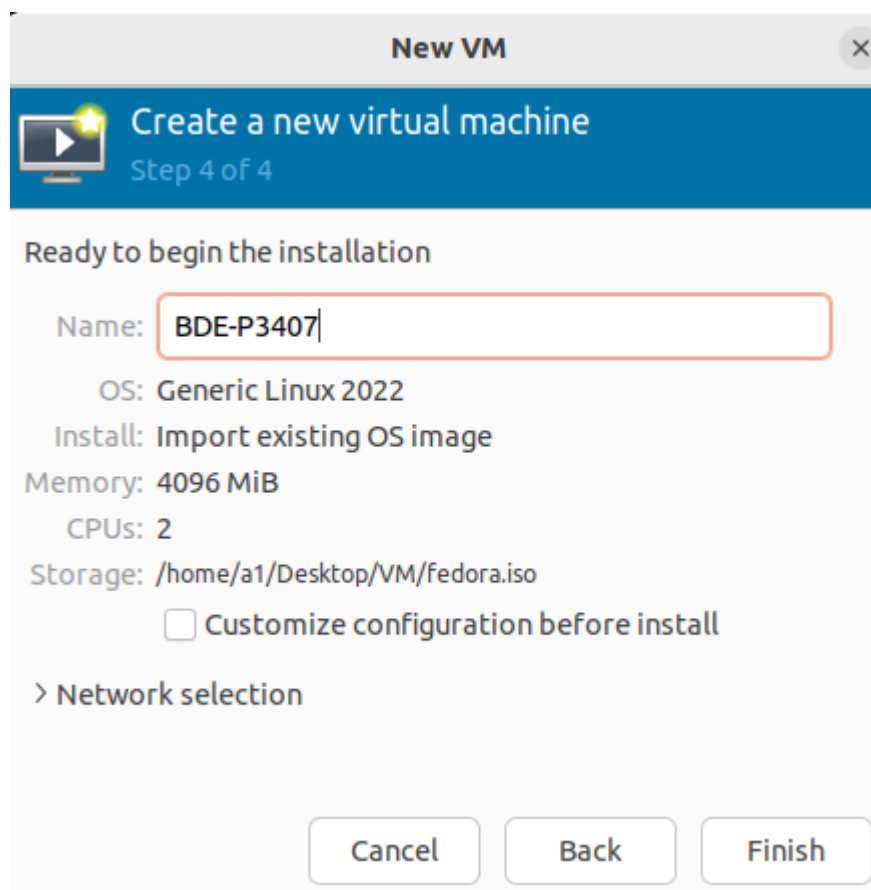


Рисунок 20 – Создание новой ВМ

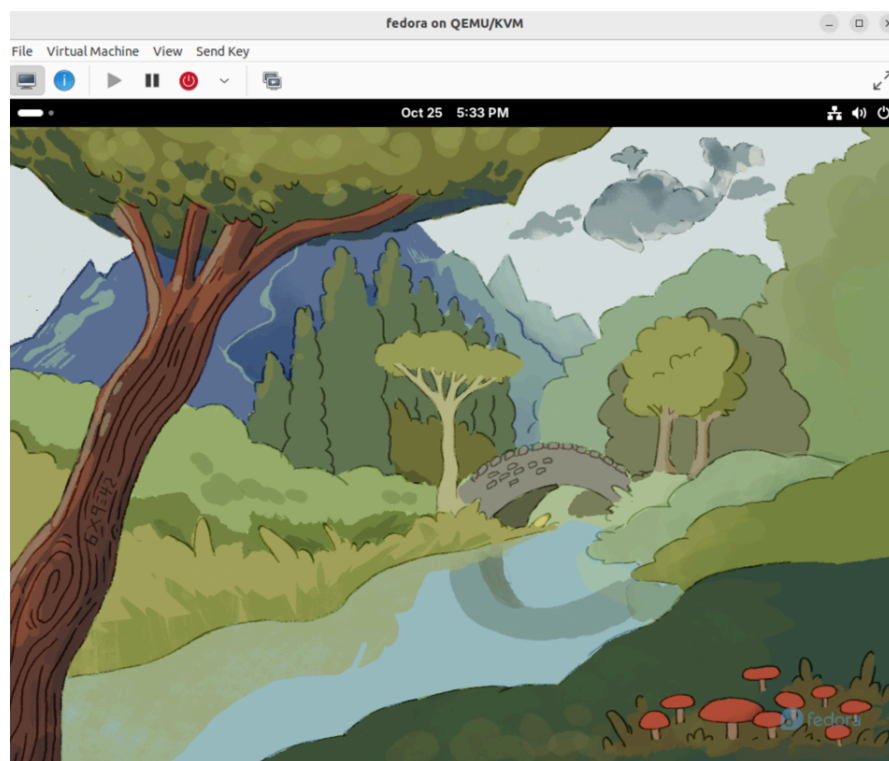


Рисунок 21 – Созданная ВМ в запущенном состоянии

Были проверены настройки системы (Рисунок 22)

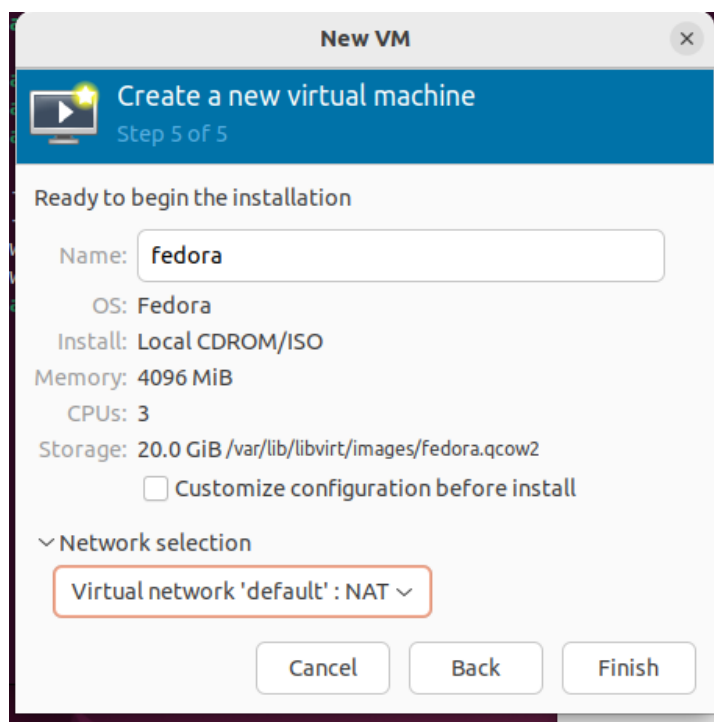


Рисунок 22 – Проверка настроек системы

Далее во вложенной ВМ был создан дополнительный жесткий диск объемом 10 МиБ, что эквивалентно 0.01 ГиБ (Рисунок 23):

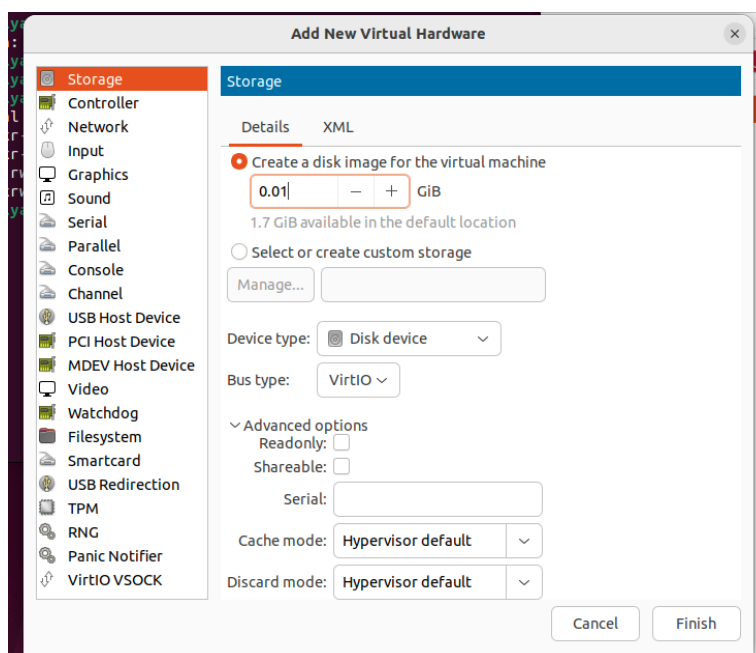


Рисунок 23 – Добавление нового жесткого диска

Был создан снимок состояния системы в момент до установки утилит (Рисунок 24).

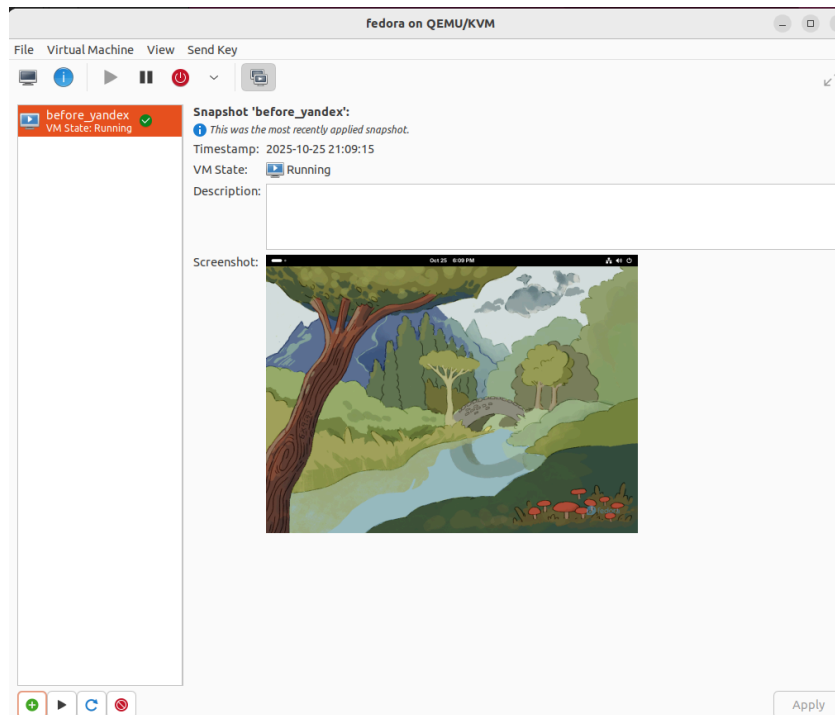


Рисунок 24 – Созданный снимок системы

Вместо Yandex был установлен htop – системный монитор в связи с нехваткой места на созданной виртуальной машине (Рисунок 25 и Рисунок 26).

```
liveuser@localhost-live:~/Downloads$ sudo dnf install htop -y
Updating and loading repositories:
Fedora 42 openh264 (From Cisco) - x86_ 100% | 3.8 KiB/s | 5.8 KiB | 00m02s
Fedora 42 - x86_64 - Updates 100% | 5.6 MiB/s | 9.2 MiB | 00m02s
Fedora 42 - x86_64 100% | 6.9 MiB/s | 35.4 MiB | 00m05s
Repositories loaded.
Package Arch Version Repository Size
Installing:
htop x86_64 3.4.1-1.fc42 updates 456.4 KiB
Installing dependencies:
hwloc-libs x86_64 2.12.0-1.fc42 fedora 2.9 MiB

Transaction Summary:
Installing: 2 packages

Total size of inbound packages is 2 MiB. Need to download 2 MiB.
After this operation, 3 MiB extra will be used (install 3 MiB, remove 0 B).
[1/2] htop-0:3.4.1-1.fc42.x86_64 100% | 660.3 KiB/s | 200.1 KiB | 00m00s
[2/2] hwloc-libs-0:2.12.0-1.fc42.x86_64 100% | 3.7 MiB/s | 2.1 MiB | 00m01s
-----
[2/2] Total 100% | 1.7 MiB/s | 2.3 MiB | 00m01s
Running transaction
[1/4] Verify package files 100% | 222.0 B/s | 2.0 B | 00m00s
[2/4] Prepare transaction 100% | 11.0 B/s | 2.0 B | 00m00s
[3/4] Installing hwloc-libs-0:2.12.0-1. 100% | 262.0 MiB/s | 2.9 MiB | 00m00s
[4/4] Installing htop-0:3.4.1-1.fc42.x8 100% | 15.9 KiB/s | 458.5 KiB | 00m29s
Complete!
```

Рисунок 25 – Установка htop

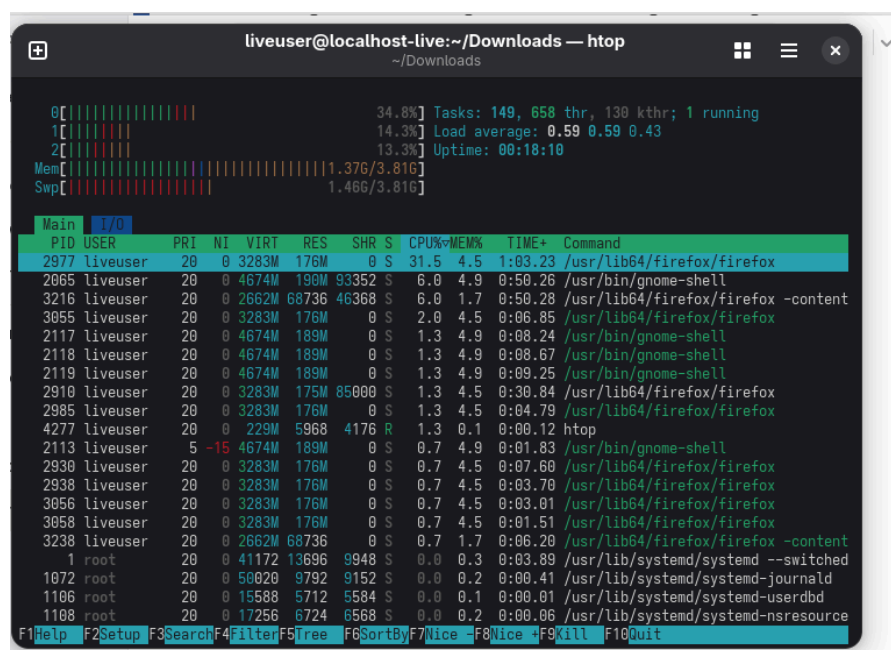


Рисунок 26 – Работающий htop

После этого состояние системы было восстановлено до первоначального и проверено наличие утилиты. Как видно, htop не установлен, а значит восстановление состояния прошло корректно (Рисунок 27):

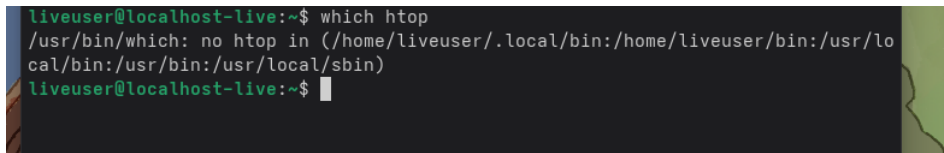


Рисунок 27 – Демонстрация отсутствия htop в системе

Также были проанализированы доступные для установки настройки ВМ (Рисунок 28).



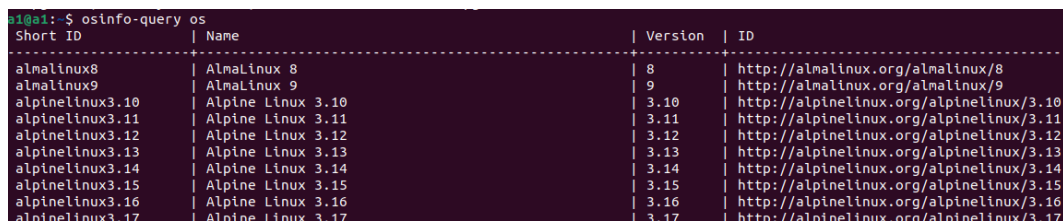
Рисунок 28 – Доступные сетевые настройки

5. Использование утилиты virt-install

Были установлены нужные пакеты с использованием актуальной команды virtinst:

```
sudo apt install virtinst libosinfo-bin
```

Проверяем, какие ОС доступны из репозитория (Рисунок 30):

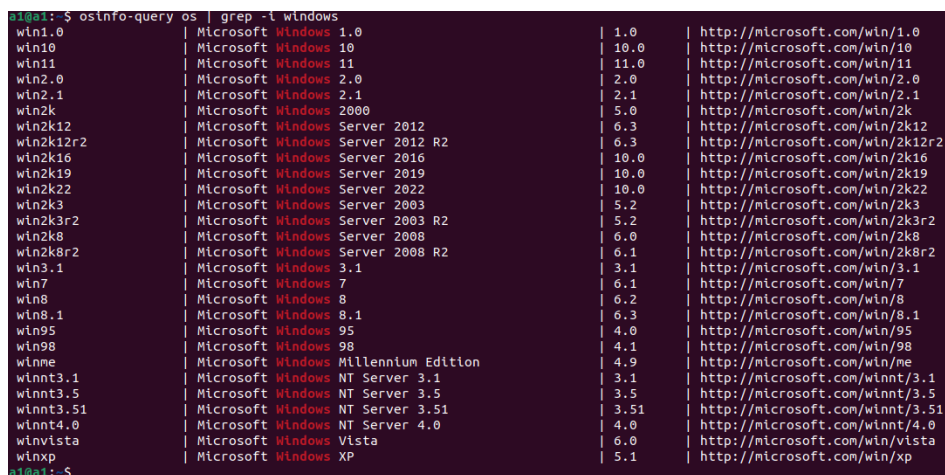


```
al@a1:~$ osinfo-query os
```

Short ID	Name	Version	ID
almalinux8	AlmaLinux 8	8	http://almalinux.org/almalinux/8
almalinux9	AlmaLinux 9	9	http://almalinux.org/almalinux/9
alpinelinux3.10	Alpine Linux 3.10	3.10	http://alpinelinux.org/alpinelinux/3.10
alpinelinux3.11	Alpine Linux 3.11	3.11	http://alpinelinux.org/alpinelinux/3.11
alpinelinux3.12	Alpine Linux 3.12	3.12	http://alpinelinux.org/alpinelinux/3.12
alpinelinux3.13	Alpine Linux 3.13	3.13	http://alpinelinux.org/alpinelinux/3.13
alpinelinux3.14	Alpine Linux 3.14	3.14	http://alpinelinux.org/alpinelinux/3.14
alpinelinux3.15	Alpine Linux 3.15	3.15	http://alpinelinux.org/alpinelinux/3.15
alpinelinux3.16	Alpine Linux 3.16	3.16	http://alpinelinux.org/alpinelinux/3.16
alpinelinux3.17	Alpine Linux 3.17	3.17	http://alpinelinux.org/alpinelinux/3.17

Рисунок 30 – Все доступные ОС

Сортируем все ОС Windows и сохраняем список в файл (Рисунок 31).



```
al@a1:~$ osinfo-query os | grep -i windows
```

Short ID	Name	Version	ID
wln1.0	Microsoft Windows 1.0	1.0	http://microsoft.com/wln/1.0
wln10	Microsoft Windows 10	10.0	http://microsoft.com/wln/10
wln11	Microsoft Windows 11	11.0	http://microsoft.com/wln/11
wln2.0	Microsoft Windows 2.0	2.0	http://microsoft.com/wln/2.0
wln2.1	Microsoft Windows 2.1	2.1	http://microsoft.com/wln/2.1
wln2k	Microsoft Windows 2000	5.0	http://microsoft.com/wln/2k
wln2k12	Microsoft Windows Server 2012	6.3	http://microsoft.com/wln/2k12
wln2k12r2	Microsoft Windows Server 2012 R2	6.3	http://microsoft.com/wln/2k12r2
wln2k16	Microsoft Windows Server 2016	10.0	http://microsoft.com/wln/2k16
wln2k19	Microsoft Windows Server 2019	10.0	http://microsoft.com/wln/2k19
wln2k22	Microsoft Windows Server 2022	10.0	http://microsoft.com/wln/2k22
wln2k3	Microsoft Windows Server 2003	5.2	http://microsoft.com/wln/2k3
wln2k3r2	Microsoft Windows Server 2003 R2	5.2	http://microsoft.com/wln/2k3r2
wln2k8	Microsoft Windows Server 2008	6.0	http://microsoft.com/wln/2k8
wln2k8r2	Microsoft Windows Server 2008 R2	6.1	http://microsoft.com/wln/2k8r2
wln3.1	Microsoft Windows 3.1	3.1	http://microsoft.com/wln/3.1
wln7	Microsoft Windows 7	6.1	http://microsoft.com/wln/7
wln8	Microsoft Windows 8	6.2	http://microsoft.com/wln/8
wln8.1	Microsoft Windows 8.1	6.3	http://microsoft.com/wln/8.1
wln95	Microsoft Windows 95	4.0	http://microsoft.com/wln/95
wln98	Microsoft Windows 98	4.1	http://microsoft.com/wln/98
wlnme	Microsoft Windows Millennium Edition	4.9	http://microsoft.com/wln/me
wlnnt3.1	Microsoft Windows NT Server 3.1	3.1	http://microsoft.com/wlnnt/3.1
wlnnt3.5	Microsoft Windows NT Server 3.5	3.5	http://microsoft.com/wlnnt/3.5
wlnnt3.51	Microsoft Windows NT Server 3.51	3.51	http://microsoft.com/wlnnt/3.51
wlnnt4.0	Microsoft Windows NT Server 4.0	4.0	http://microsoft.com/wlnnt/4.0
wlnvista	Microsoft Windows Vista	6.0	http://microsoft.com/wln/vista
wlnxp	Microsoft Windows XP	5.1	http://microsoft.com/wln/xp



```
al@a1:~$ cat windows-list.txt
```

Short ID	Name	Version	ID
wln1.0	Microsoft Windows 1.0	1.0	http://microsoft.com/wln/1.0
wln10	Microsoft Windows 10	10.0	http://microsoft.com/wln/10
wln11	Microsoft Windows 11	11.0	http://microsoft.com/wln/11
wln2.0	Microsoft Windows 2.0	2.0	http://microsoft.com/wln/2.0
wln2.1	Microsoft Windows 2.1	2.1	http://microsoft.com/wln/2.1
wln2k	Microsoft Windows 2000	5.0	http://microsoft.com/wln/2k

Рисунок 31 – Доступные ОС Windows

Сортируем все ОС Fedora и также сохраняем список в файл (Рисунок 32).

```

a1@a1: $ osinfo-query os | grep -i fedora
fedora-coreos-next | Fedora CoreOS | Next | http://fedoraproject.org/coreos/next
fedora-coreos-stable | Fedora CoreOS | Stable | http://fedoraproject.org/coreos/stable
fedora-coreos-testing | Fedora CoreOS | Testing | http://fedoraproject.org/coreos/testing
fedora-elx | Fedora ELN | ELN | http://fedoraproject.org/fedora/elx
fedora-rawhide | Fedora Rawhide | Rawhide | http://fedoraproject.org/fedora/rawhide
fedora-unknown | Fedora | unknown | http://fedoraproject.org/fedora/unknown
fedora1 | Fedora 1 | 1 | http://fedoraproject.org/fedora/1
fedora10 | Fedora 10 | 10 | http://fedoraproject.org/fedora/10
fedora11 | Fedora 11 | 11 | http://fedoraproject.org/fedora/11
fedora12 | Fedora 12 | 12 | http://fedoraproject.org/fedora/12

a1@a1: $ osinfo-query os | grep -i fedora > fedora-list.txt
a1@a1: $ cat fedora-list.txt
fedora-coreos-next | Fedora CoreOS | Next | http://fedoraproject.org/coreos/next
fedora-coreos-stable | Fedora CoreOS | Stable | http://fedoraproject.org/coreos/stable
fedora-coreos-testing | Fedora CoreOS | Testing | http://fedoraproject.org/coreos/testing
fedora-elx | Fedora ELN | ELN | http://fedoraproject.org/fedora/elx
fedora-rawhide | Fedora Rawhide | Rawhide | http://fedoraproject.org/fedora/rawhide
fedora-unknown | Fedora | unknown | http://fedoraproject.org/fedora/unknown

```

Рисунок 32 – Доступные ОС Fedora

6. Добавление ВМ с помощью virt-install

Для добавления ВМ через командную строку используем следующую команду:

```

virt-install --name bde-p3407 --virt-type=kvm --memory 4096
--vcpus=2 --os-variant=archlinux --hvm
--cdrom=/home/a1/Downloads/archlinux-x86_64.iso --network
default,model=virtio --disk
path=/home/a1/Downloads/archlinux.qcow2,size=8,bus=virtio,format
=qcow2 --graphics vnc --noautoconsole

```

В команде используются следующие параметры:

Основные параметры виртуальной машины:

--name bde-p3407 — задает имя виртуальной машины, которое будет использоваться для идентификации в системе libvirt.

--virt-type=kvm — определяет тип гипервизора. Значение kvm задает использование модуля ядра KVM для аппаратной виртуализации, для обеспечения максимальной производительности.

Конфигурация аппаратных ресурсов:

--memory 4096 — выделяет виртуальной машине 4096 МБ оперативной памяти.

--vcpus=2 — назначает виртуальной машине 2 виртуальных процессорных ядра.

Параметры операционной системы:

--os-variant=archlinux — указывает тип устанавливаемой операционной системы. libvirt использует эту информацию для автоматической оптимизации параметров виртуального оборудования под конкретный дистрибутив.

--hvm — активирует режим полной виртуализации (Hardware Virtual Machine), при котором гостевая система получает доступ к виртуализированным аппаратным компонентам и может выполнять привилегированные инструкции.

Установочный носитель:

--cdrom=/home/a1/Downloads/archlinux-x86_64.iso — определяет путь к ISO-образу устанавливаемой системы.

Конфигурация сети:

--network default,model=virtio — настраивает сетевой интерфейс виртуальной машины. Параметр default активирует использование стандартной NAT-сети libvirt. Параметр model=virtio определяет тип сетевого адаптера.

Конфигурация дискового пространства:

--disk path=/home/a1/Downloads/archlinux.qcow2,size=8,bus=virtio,format=qcow2 — определяет параметры виртуального жесткого диска: path — путь к файлу образа диска, size=8 — размер виртуального диска 8 ГБ, bus=virtio — тип шины подключения диска (virtio обеспечивает высокую производительность операций ввода-вывода), format=qcow2 — формат образа диска QCOW2 (QEMU Copy-On-Write)

Параметры графического вывода:

--graphics vnc — настраивает графическую консоль виртуальной машины с использованием протокола VNC (Virtual Network Computing). Это позволяет подключаться к графическому интерфейсу VM удаленно или через утилиту virt-viewer.

--noautoconsole — отключает автоматическое открытие консоли виртуальной машины после её создания, то есть VM запускается в фоновом режиме

Проверили запуск виртуальной машины, запуск прошел успешно (Рисунок 33):

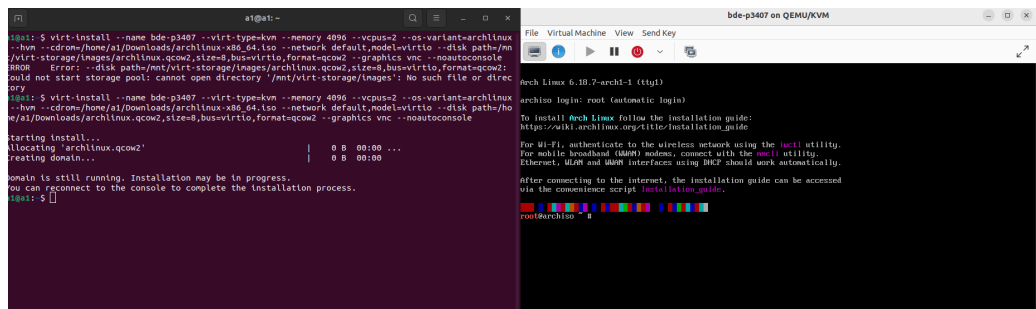


Рисунок 33 – Проверка запуска ВМ

Запуск машины в фоновом режиме осуществляется посредством использования параметра `--noautoconsole` в команде добавления ВМ.

Далее подключаемся к созданной ВМ через SSH. Для этого нужно установить `openssh-server` на вложенную ОС, запустить `ssh` (Рисунок 34)

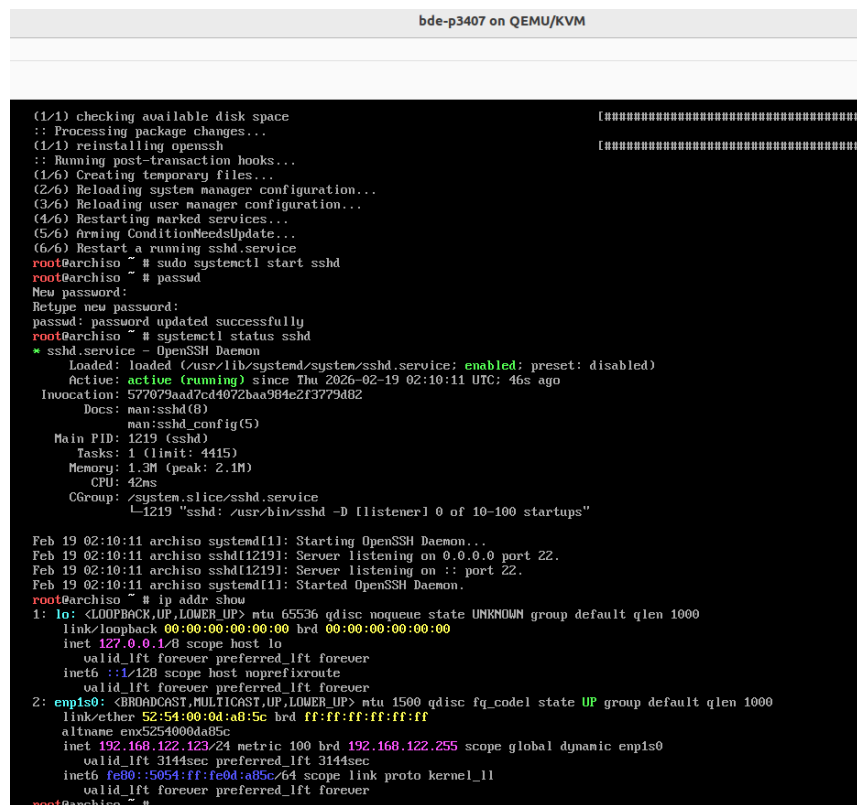


Рисунок 34 – Установка и запуск ssh сервера

После чего подключиться к ней с гостевой ОС 1 (Ubuntu), используя логин и пароль live юзера (Рисунок 35):

```
a1@a1:~$ ssh root@192.168.122.123
root@192.168.122.123's password:
To install Arch Linux follow the installation guide:
https://wiki.archlinux.org/title/Installation_guide

For Wi-Fi, authenticate to the wireless network using the iwctl utility.
For mobile broadband (WWAN) modems, connect with the mmcli utility.
Ethernet, WLAN and WWAN interfaces using DHCP should work automatically.

After connecting to the internet, the installation guide can be accessed
via the convenience script Installation_guide.

root@archiso ~ #
```

Рисунок 35 – Подключение по ssh к созданной ВМ

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было развернуто окружение для вложенной виртуализации в операционной системе Ubuntu с подключением дополнительного виртуального диска объемом 30 ГБ для размещения образов виртуальных машин.

Освоены три подхода к созданию и управлению виртуальными машинами. С использованием Gnome Boxes развернута VM в Live-режиме для изучения принципов быстрого запуска тестовых окружений. Посредством прямой работы с QEMU/KVM создана виртуальная машина Fedora с ручной настройкой параметров, включая конфигурацию сетевого адаптера с заданным MAC-адресом и режим полноэкранного отображения. С помощью Virtual Machine Manager развернута полнофункциональная VM с управлением через графический интерфейс.

Изучены механизмы создания снимков состояния. Создан снимок начальной конфигурации системы, выполнена установка дополнительного ПО (htop) и осуществлено восстановление к исходному состоянию. Проверка подтвердила корректность работы — установленное приложение отсутствовало после восстановления.

Освоена процедура клонирования виртуальных машин с использованием утилиты virt-clone для создания шаблонных систем.

Получены практические навыки работы с консольными утилитами libvirt. Через virsh выполнены операции управления жизненным циклом VM (запуск, остановка, автозапуск). С помощью virt-install создана виртуальная машина с заданными параметрами. Проведена фильтрация списков поддерживаемых операционных систем семейств Windows и Fedora с сохранением результатов.

Настроено удаленное RDP-подключение из вложенной виртуальной машины в изолированной NAT-сети к физическому компьютеру через проброс портов с использованием утилиты socat. Проверено успешное установление соединения.

Исследованы возможности настройки виртуального оборудования в Virtual Machine Manager: изменение ресурсов, добавление дисков, конфигурация сетевых адаптеров. Изучены параметры мониторинга VM в реальном времени: загрузка процессора, использование памяти, дисковая активность и сетевой трафик.

В результате работы получены практические навыки развертывания и администрирования виртуальных машин в Linux-системах с использованием QEMU/KVM и инструментария libvirt, освоены методы графического управления и автоматизации через консольные утилиты.